

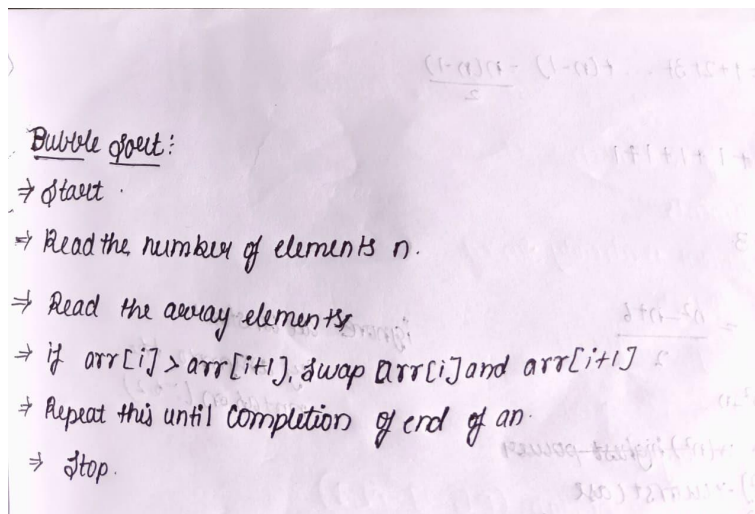


PRACTICAL: 01

Aim: To implement and find the time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quick sort.

A) Bubble Sort:

Algorithm:



Program / Code:

```
#include<iostream>

using namespace std;

int main()
{
    int a[60], size, i, j, temp;
    cout<<"Enter the size of the array: ";
    cin>>size;
    cout<<"Enter the array elements: ";
    for(i=0; i<size; i++){
        cin>>a[i];
    }
    for(i=0; i<size-1; i++){
```



```
for(j=0; j<size-i-1; j++){  
    if(a[j]>a[j+1]){  
        temp = a[j];  
        a[j] = a[j+1];  
        a[j+1] = temp;  
    }  
}  
}  
  
cout<<"The Elements are: ";  
for(i=0; i<size; i++){  
    cout<<a[i];  
}  
  
return 0;  
}
```

Output:

```
*C:\Users\karth\Downloads\D x + v  
Enter the size of the array: 4  
Enter the array elements: 3  
6  
9  
2  
The Elements are: 2369  
Process returned 0 (0x0)   execution time : 8.906 s  
Press any key to continue.
```



Time Complexity:

Time complexity

```

for (int i = 0; i < n-1; i++) ... outer loop
{
    for (int j = 0; j < n-1-i; j++) ... inner loop
    {
        if (arr[i] > arr[j]) ...
        {
            int temp = arr[j];
            arr[j+1] = arr[i];
            arr[i] = temp;
        }
    }
}

```

→ Outer loop runs i=0 to n-1
So it executes n-1 times

→ Inner loop: pass(i) iterations

Diagram showing iterations for inner loop:

0	n-1
1	n-2
2	...
n-2	1

$$\begin{aligned}
 (n-1) + (n-2) + \dots + 2 + 1 &= 1 + 2 + 3 + \dots + (n-1) = \frac{n(n-1)}{2} \\
 T(n) &= n-1 + \frac{n^2-n}{2} + 1 + 1 + 1 \\
 &= \frac{n^2-n}{2} + 3 \\
 &= \frac{2n + n^2 - n + 6}{2} = \frac{n^2 - n + 6}{2} \\
 &= n^2 - n
 \end{aligned}$$

ignore all constant by asymptotic notation ($\sim$)

$T(n) = O(n^2)$ highest power

$T(n) = \Theta(n^2)$ - worst case

$T(n) = O(n^2)$ - in average case

$T(n) = \Theta(n)$ - Best case

B) Insertion Sort:

Algorithm:

INSERTION SORT

- Start
- Read the number of element n
- Read the array elements
- Assume the first element is already sorted
- Repeat for i=1 to n-1:
 - Set key = A[i]
 - Set j = i-1
 - while j > 0 and A[j] > key.
 - * move A[j] one position to the right (A[j+1] = A[j])
 - * Decrement j.
 - * Insert key at position j+1
- After all passes the array is sorted
- Display the sorted array.
- Stop.



Program / Code:

```
#include<iostream>

using namespace std;

int main(){

    int a[60], size, i, j, min, temp;

    cout<<"Enter the size of the array: ";

    cin>>size;

    cout<<"Enter the array elements: ";

    for(i=0; i<size; i++){

        cin>>a[i];

    }

    for(i=0; i<size-1; i++){

        min = i;

        for(j=i+1; j<size; j++){

            if(a[j] < a[min]){

                min = j;

            }

        }

        if(min != i){

            temp = a[i];

            a[i] = a[min];

            a[min] = temp;

        }

    }

    cout<<"The Elements are: ";

    for(i=0; i<size; i++){

        cout<<a[i]<<" ";

    }

}
```

```

    }

    return 0;

}

```

Output:

```

C:\Users\karth\Downloads\ID > + >
Enter the size of the array: 4
Enter the array elements: 6
3
9
2
The Elements are: 2 3 6 9
Process returned 0 (0x0)   execution time : 19.363 s
Press any key to continue.

```

Time Complexity:

Time complexity:

Insertion sort

```

for (int i = 1; i < n; i++)
{
    int key = arr[i];
    int j = i - 1;
    while (j >= 0 & arr[j] > key)
    {
        arr[j + 1] = arr[j];
        j--;
    }
    arr[j + 1] = key;
}

```

Best case: Already sorted array
[10, 20, 30, 40, 50] → outer loop runs $n-1$ times
→ while checks only once for each element
 $T(n) = O(n)$



C) Selection Sort:

Algorithm:

2) Selection Sort :

- ⇒ Start
- Read the number of elements n .
- Read the array elements
- Repeat for $i=0$ to $n-1$:
 - * Set $min=i$
 - * Repeat for $j=i+1$ to $n-1$:
 - if $A[j] < A[min]$ then set $min=j$
 - * if $min \neq i$, swap $A[i]$ and $A[min]$
- ⇒ After all passes the array is sorted in ascending order
- ⇒ Display the sorted array
- ⇒ Stop.

Program / Code:

```
#include <iostream>
using namespace std;
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        int temp = arr[minIndex];
```



```
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    selectionSort(arr, n);
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++){
        cout << arr[i] << " ";
    }
    return 0;
}
```

Output:

Time complexity:

Time complexity:

Selection Sort:

```

for (int i = 0; i < n-1; i++) ... n-1
{
    int minIndex = i; ... 1
    for (int j = i+1; j < n; j++) ...  $\frac{n^2-n}{2}$ 
    {
        if (arr[j] < arr[minIndex]) ... 1
        {
            minIndex = j; ... 1
        }
    }
    int temp = arr[minIndex];
    arr[minIndex] = arr[i];
    arr[i] = temp;
}

```

Outer loop: The outer loop runs from $i=0$ to $n-1$
so it executes $n-1$ times

Inner loop: pass comparisons

1	$n-1$	$(n-1) + (n-2) + (n-3) + \dots + 2 + 1$
2	$n-2$	
3	$n-3$	
$\vdots$	$\vdots$	
$n-1$	1	

$T(n) = n-1 + \frac{n^2-n}{2} + 1 + 1 + 1 + 1$

$= \frac{n + n^2 - n}{2} + 4 = \frac{n^2}{2} + n + 4$

$= 4n^2 + n + 4$

$= n^2 + n$ (by assumption to notation avoid const)

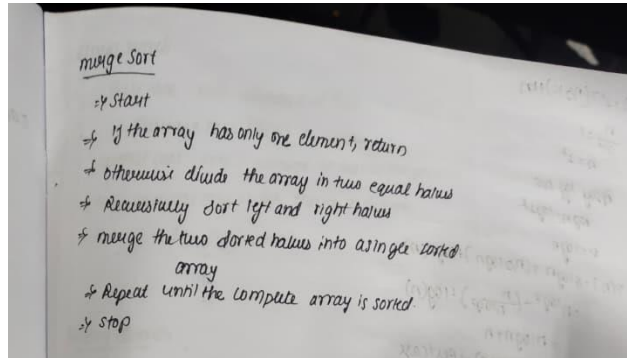
$T(n) = O(n^2)$ (highest power)

$T(n) = O(n^2)$ — worst case
 $T(n) = O(n^2)$ — best case
 $T(n) = O(n^2)$ — Average case



D) Merge Sort:

Algorithm:



Program / Code:

```
#include <iostream>

using namespace std;

void merge(int arr[], int left, int mid, int right) {

    int n1 = mid - left + 1;

    int n2 = right - mid;

    int L[n1], R[n2];

    for (int i = 0; i < n1; i++) {

        L[i] = arr[left + i];

    }

    for (int j = 0; j < n2; j++) {

        R[j] = arr[mid + 1 + j];

    }

    int i = 0;

    int j = 0;

    int k = left;

    while (i < n1 && j < n2) {

        if (L[i] <= R[j]) {

            arr[k++] = L[i++];

        }
```



```
        } else {
            arr[k++] = R[j++];
        }
    }
    while (i < n1) {
        arr[k++] = L[i++];
    }
    while (j < n2) {
        arr[k++] = R[j++];
    }
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
}
```



```
}  
  
mergeSort(arr, 0, n - 1);  
  
cout << "Sorted array using Merge Sort: ";  
for (int i = 0; i < n; i++) {  
    cout << arr[i] << " ";  
}  
  
cout << endl;  
  
return 0;  
}
```

Output:

```
"C:\Users\karth\Downloads\D" x + v  
Enter number of elements: 4  
Enter elements: 2  
3  
6  
4  
Sorted array using Merge Sort: 2 3 4 6  
Process returned 0 (0x0) execution time : 8.211 s  
Press any key to continue.  
|
```



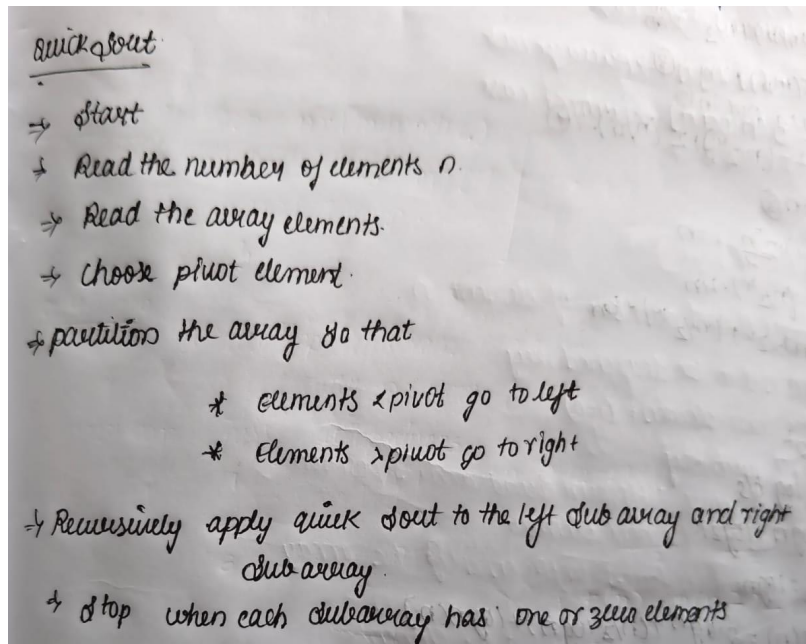
Time Complexity:

merge sort
The array is divided into two halves repeatedly
No. of divisions : $\log_2 (2T(n/2))$
+ merge each level merges all elements
 $O(n)$
Total time $T(n) = 2T(n/2) + n$
 $T(n) = 2T(n/2) + n \rightarrow \text{---} \text{---} \text{---}$
 $n \rightarrow n/2$ put in ---
 $T(n/2) = 2T(n/4) + n/2$
sub --- in ---
 $T(n) = 2(2T(n/4) + n/2) + n$
 $= 2T(n/4) + 2n + n$
 $= 2T(n/4) + 3n$
 $n \rightarrow n/4$ put in ---
 $T(n/2) = 2T(n/4) + 2(n/2) + \text{---}$
 --- in ---
 $T(n) = 2^2 [2T(n/2^4) + \frac{2n}{4}] + 2n$
 $= 2^4 T(n/2^4) + \frac{2^3 n}{4} + 2n$
 $= 2^4 T(n/2^4) + 4n$
 $T(n) = 2^k T(n/2^k) + kn$

$T(n) = 2^k T(n/2^k) + kn$
 $\frac{n}{2^k} = 1$
 $n = 2^k$
Apply log b/s
 $\log n = \log 2^k$
 $k = \log n$
 $T(n) = 2 \log n + (n/2 \log n) + \log n(n)$
 $= n \log^2 - (\frac{n}{n \log^2}) + \log(n)$
 $= n \log n + n$
 $T(n) = O(n \log n) \rightarrow \text{Best case}$
 $T(n) = O(n \log n) \rightarrow \text{Average case}$
 $T(n) = O(n \log n) \rightarrow \text{Worst case}$

E) Quick Sort:

Algorithm:



Program / Code:

```
#include <iostream>

using namespace std;

int partition(int arr[], int lb, int ub) {
    int pivot = arr[lb];
    int start = lb;
    int end = ub;
    while (start < end) {
        while (arr[start] <= pivot) {
            start++;
        }
        while (arr[end] > pivot) {
            end--;
        }
    }
}
```



```
    }  
    if (start < end) {  
        swap(arr[start], arr[end]);  
    }  
}  
swap(arr[lb], arr[end]);  
return end;  
}  
void quickSort(int arr[], int lb, int ub) {  
    if (lb < ub) {  
        int loc = partition(arr, lb, ub);  
        quickSort(arr, lb, loc - 1);  
        quickSort(arr, loc + 1, ub);  
    }  
}  
int main() {  
    int n;  
    cout << "Enter size of array: ";  
    cin >> n;  
    int arr[n];  
    cout << "Enter " << n << " elements: ";  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
    quickSort(arr, 0, n - 1);  
    cout << "Sorted array using Quick Sort: ";  
    for (int i = 0; i < n; i++) {
```



```
        cout << arr[i] << " ";  
    }  
    return 0;  
}
```

Output:

```
"C:\Users\karth\Downloads\ID" X + v  
Enter size of array: 4  
Enter 4 elements: 9  
3  
5  
6  
Sorted array using Quick Sort: 3 5 6 9  
Process returned 0 (0x0) execution time : 9.644 s  
Press any key to continue.  
|
```

Time Complexity:

Quick sort
→ the pivot divides the array into two equal halves → $T(n/2)$
→ for floating we use n elements at last

$$T(n) = 2T(n/2) + n$$

$$T(n) = 2T(n/2) + n \rightarrow (1)$$

$$n \rightarrow n/2 \text{ put in } (1)$$

$$T(n/2) = 2T(n/4) + n/2$$

$$T(n) = 2T(n/4) + n/2 \rightarrow (2)$$

sub in (1)

$$T(n) = 2(2T(n/4) + n/2) + n$$

$$= 2^2 T(n/4) + 2n \rightarrow (3)$$

$$n \rightarrow n/4 \text{ put in } (3)$$

$$T(n/4) = 2T(n/8) + n/4 \rightarrow (4)$$

sub in (3)

$$T(n) = 2^3 T(n/8) + 2(n/2) + 2(n/4) + n$$

$$= 2^k T(n/2^k) + 2 \frac{n}{2} + 2 \frac{n}{4} + \dots + n$$

$$T(n) = 2^k T(n/2^k) + kn$$

$\frac{n}{2^k} = 1$
 $n = 2^k$
Apply log b/s
 $\log n = \log 2^k$
 $k = \log n$

$$T(n) = 2^{\log n} T(n/2^{\log n}) + (\log n) n$$

$$= n^{\log 2} (n/n^{\log 2}) + \log(n)$$

$$= n \log n + n$$

$$T(n) = O(n \log n)$$

$T(n) = O(n \log n) \rightarrow \text{Best case}$
 $T(n) = O(n \log n) \rightarrow \text{Average case}$
 $T(n) = O(n^2) \rightarrow \text{Worst case}$

Conclusion:

In this practical, we studied and implemented five sorting algorithms: Bubble, Selection, and Insertion Sort are simple but slow for large data ($O(n^2)$ time), making them suitable only for small lists. Merge Sort and Quick Sort are much faster for large data since they run in $O(n \log n)$ time. So, simple sorts work fine for small lists, but Merge/Quick Sort are better choices for big data.